DISCUSSION DOCUMENT

Prepared by: **Anuska Nath**

Roll No: **002311001003**

Assignment 2 - Brief

Construct a machine learning based model for classification using Python for the following UCI datasets:

- **Wine Dataset** (archive.ics.uci.edu/ml/machine-learning-databases/wine/wine.data)
- **Handwritten Digit Dataset** (archive.ics.uci.edu/ml/datasets/Optical+Recognition+of+Handwritten+Digits)

Task 1: Employ SVM classifier using Linear, Polynomial, Gaussian/RBF and Sigmoid kernels and show classification results (Accuracy, Precision, Recall, F-score, confusion matrix) with and without parameter tuning.

Task 2: Use MLP classifier and Random Forest classifier for both datasets. Tune the required parameters and show classification results (Accuracy, Precision, Recall, F-score, confusion matrix), along with training/loss curves.

Task 3: Apply different train-test splits (50:50, 60:40, 70:30 and 80:20) and compare the performance of all three classifiers. Generate confusion matrix heatmaps and ROC curves with AUC for each dataset.

Task 4: Apply PCA for feature dimensionality reduction and again use SVM, MLP and Random Forest. Compare the results with the original feature set and prepare a final performance comparison table. Try to achieve an accuracy of 90% or above through parameter tuning.

Assignment Link

1. Wine Dataset

1.1 Dataset Description

The Wine dataset contains **178 samples, 13 features and 3 classes**. It is used to classify wines based on their chemical properties.

1.2 Data Preprocessing

The dataset is loaded and separated into **features (X)** and **target classes (y)**. The data is divided into training and testing sets using different train-test ratios. **Feature scaling** is applied for SVM and MLP classifiers.

2. Handwritten Digits Dataset

2.1 Dataset Description

The Handwritten Digits dataset contains **handwritten digit images from 0 to 9**, represented using **64 features**. It is used for **10-class digit classification**.

2.2 Data Preprocessing

The dataset is divided into **features (X)** and **target classes (y)**. The data is split into training and testing sets using different train-test ratios. **Feature scaling** is applied for SVM and MLP classifiers.

3. Evaluation Metrics

The performance of the classifiers is evaluated using the following metrics:

- **Accuracy** – Percentage of correctly classified samples.
- **Precision** – Measures how many predicted positive samples are actually correct.
- **Recall** – Measures how many actual positive samples are correctly identified.
- **F-score (F1-score)** – Harmonic mean of Precision and Recall.
- **Confusion Matrix** – Shows the correct and incorrect predictions for each class. ● **ROC-AUC** – Measures the classifier's ability to distinguish between different classes.

4. Support Vector Machine (SVM)

Support Vector Machine is a supervised classification algorithm that finds the best boundary between different classes. Different kernels are used to handle linear and non-linear data.

4.1 Linear Kernel

The Linear kernel is used when the classes can be separated using a straight decision boundary.

Succeeds / Fails

Where it succeeds	Where it fails
● Simple and fast to train.	● Cannot handle complex non-linear boundaries.
● Works well when data is approximately linearly separable.	● May give lower accuracy when classes overlap.
● Easy to interpret compared with non-linear kernels.	● Performance can decrease if the data has complex patterns.

4.2 Polynomial Kernel

The Polynomial kernel creates a curved decision boundary using polynomial functions.

Succeeds / Fails

Where it succeeds	Where it fails
● Can handle non-linear relationships.	● Higher polynomial degrees can cause overfitting.
● Useful when the class boundary has a polynomial pattern.	● Training can be slower than Linear SVM.
● Degree can be adjusted to improve performance.	● Performance is sensitive to the choice of degree and other parameters.

4.3 Gaussian/RBF Kernel

The Gaussian or RBF kernel is commonly used for non-linear classification and can create flexible decision boundaries.

Succeeds / Fails

Where it succeeds	Where it fails
● Handles complex non-linear relationships well.	● Requires proper selection of `C` and gamma.
● Usually performs well on datasets with overlapping classes.	● Can be slower than Linear SVM for larger datasets.
● Often provides high classification accuracy after tuning.	● Poor parameter selection may lead to overfitting or underfitting.

4.4 Sigmoid Kernel

The Sigmoid kernel uses a sigmoid function to create the classification boundary.

Succeeds / Fails

Where it succeeds	Where it fails
● Can handle some non-linear relationships.	● Often less accurate than RBF for complex datasets.
● Provides an alternative to RBF and Polynomial kernels.	● Sensitive to parameter selection.
● Can perform well with suitable parameter values.	● May perform poorly when the data is not suitable for a sigmoid boundary.

4.5 Parameter Tuning

The main SVM parameters are tuned to improve classification performance.

- **C** – Controls the trade-off between training errors and model complexity.
- **Gamma** – Controls the influence of individual training points for non-linear kernels.
- **Degree** – Controls the degree of the Polynomial kernel.
- **Kernel** – Linear, Polynomial, RBF and Sigmoid are compared.

Succeeds / Fails

Where it succeeds	Where it fails
● Helps improve classification accuracy.	● Requires additional training time.
● Finds better parameter combinations.	● Too much tuning may lead to overfitting.
● Reduces the chance of using a poorly configured model.	● Results depend on the selected parameter ranges.

5.Multi-Layer Perceptron (MLP)

Multi-Layer Perceptron is a feed-forward artificial neural network that learns complex relationships between input features and class labels through interconnected layers of neurons.

5.1 Architecture

The MLP consists of an **input layer, one or more hidden layers, and an output layer**. Each neuron applies weights, bias and an activation function to the incoming data. During training, the weights are adjusted through backpropagation to minimize the classification error.

Succeeds / Fails

Where it succeeds	Where it fails
● Learns complex and non-linear patterns effectively.	● Requires careful selection of network architecture.
● Multiple hidden layers allow flexible decision boundaries.	● More computationally demanding than simpler classifiers.
● Suitable for both Wine and Handwritten Digit classification.	● Poor architecture selection can result in underfitting or overfitting.

5.2 Momentum

Momentum is used during gradient-based optimization to accelerate learning and reduce unnecessary fluctuations in the weight updates. It allows the model to retain part of the previous update while calculating the current update.

Succeeds / Fails

Where it succeeds	Where it fails
● Speeds up convergence during training.	● Very high momentum can make training unstable.
● Helps reduce oscillations during optimization.	● Requires suitable tuning for different datasets.
● Can improve stability when navigating complex loss surfaces.	● Improper values may cause the model to overshoot the optimum.

5.3 Learning Rate

The learning rate controls the size of the steps taken while updating the network weights during training.

Succeeds / Fails

Where it succeeds	Where it fails
● A suitable learning rate provides faster and more stable convergence.	● A very high learning rate may prevent convergence.
● Helps the model reach a lower training loss.	● A very low learning rate can make training excessively slow.
● Proper tuning can significantly improve classification performance.	● The optimal value depends on the dataset and network architecture.

5.4 Epochs

An epoch represents one complete pass of the training dataset through the neural network. The number of epochs determines how many times the model updates its parameters using the training data.

Succeeds / Fails

Where it succeeds	Where it fails
● More epochs can allow the model to learn sufficiently complex patterns.	● Excessive epochs may lead to overfitting.
● Helps achieve lower training loss when the model is initially undertrained.	● Too few epochs can result in underfitting.
● Appropriate epoch selection can improve overall predictive performance.	● Increasing epochs also increases training time.

5.5 Parameter Tuning

The MLP parameters are systematically varied to obtain a better-performing model. The main parameters considered are momentum, learning rate, number of epochs and hidden-layer configuration.

Succeeds / Fails

Where it succeeds	Where it fails
● Identifies a configuration that provides better classification performance.	● Hyperparameter search increases computational cost.
● Helps balance training speed, convergence and generalization.	● Poor parameter ranges may fail to find a good configuration.
● Can significantly improve the model compared with the default configuration.	● Excessive tuning can increase the risk of overfitting.

5.6 Training/Loss Curves

Training and loss curves are used to observe how the MLP learns over successive epochs. The loss curve shows whether the model's prediction error decreases during training, while training performance indicates how effectively the model learns from the data.

Succeeds / Fails

Where it succeeds	Where it fails
● Provides a clear view of the model's learning behaviour.	● A decreasing training loss does not always guarantee good test performance.
● Helps identify convergence and suitable training duration.	● Irregular curves may indicate unsuitable learning-rate or momentum settings.
● Can reveal whether the model is underfitting or overfitting.	● Very long training may improve training performance while reducing generalization.

6. Random Forest

Random Forest is an ensemble learning algorithm that combines multiple decision trees to produce a more accurate and robust classification model. It uses random samples of data and features to build diverse trees and combines their predictions through voting.

6.1 Working Principle

Random Forest creates several decision trees using different subsets of the training data and features. Each tree independently produces a prediction, and the final class is selected based on the majority vote of all trees.

Succeeds / Fails

Where it succeeds	Where it fails
● Provides a clear view of the model's learning behaviour.	● A decreasing training loss does not always guarantee good test performance.
● Helps identify convergence and suitable training duration.	● Irregular curves may indicate unsuitable learning-rate or momentum settings.
● Can reveal whether the model is underfitting or overfitting.	● Very long training may improve training performance while reducing generalization.

6.2 Parameter Tuning

Important Random Forest parameters are tuned to improve classification performance. The main parameters include **number of trees, maximum tree depth, minimum samples for splitting and minimum samples per leaf**.

Succeeds / Fails

Where it succeeds	Where it fails
● Helps identify the most suitable forest configuration.	● Extensive parameter searching increases computation time.
● Can improve accuracy and generalization.	● Very deep trees may cause overfitting.
● Controls the complexity of individual trees and the overall model.	● Too few trees may produce unstable or less reliable predictions.

6.3 Performance Curves

Performance curves are generated to study how the model behaves as training conditions or the number of trees changes. These curves help compare training and testing performance and understand model stability.

Succeeds / Fails

Where it succeeds	Where it fails
● Shows how model performance changes with the number of trees.	● More trees do not always result in a significant accuracy improvement.
● Helps identify a suitable model complexity.	● Performance may become stable after a certain number of trees.
● Makes comparison between training and testing performance easier.	● Curves can require additional experiments and computation.

7. Wine Dataset Results

7.1 Untuned Results

The SVM, MLP and Random Forest classifiers were first evaluated using their default parameters. Accuracy, Precision, Recall, F-score and Confusion Matrix were used for comparison.

Succeeds / Fails

Where it succeeds	Where it fails
● Provides a baseline for comparing the classifiers.	● Default parameters may not provide the best performance.
● Requires minimal parameter selection.	● Some SVM kernels may perform poorly without tuning.
● Helps identify which models benefit most from tuning.	● MLP performance can depend strongly on its default learning configuration.

7.2 Tuned Results

The classifiers were then tuned by testing suitable hyperparameter combinations. The best-performing configurations were selected based on their classification performance.

Succeeds / Fails

Where it succeeds	Where it fails
● Improves the performance of suitable models.	● Tuning requires additional computation.
● Helps select the best SVM kernel and model parameters.	● Not every model necessarily improves after tuning.
● Provides a stronger and more reliable comparison.	● Excessive tuning may increase the risk of overfitting.

7.3 Train-Test Split Comparison

The Wine dataset was evaluated using 50:50, 60:40, 70:30 and 80:20 train-test splits. The results were compared to study the effect of training-data size on model performance.

Succeeds / Fails

Where it succeeds	Where it fails
● Shows how classifiers perform with different amounts of training data.	● Results can vary depending on the selected split.

● Helps identify a suitable train-test ratio.	● Smaller training sets may reduce model performance.
● Provides a broader comparison of model stability.	● Different splits can produce different accuracy values.

7.4 Confusion Matrices

Confusion matrices were generated for the different classifiers to show the number of correct and incorrect predictions for each of the three Wine classes.

Succeeds / Fails

Where it succeeds	Where it fails
● Clearly shows class-wise classification performance.	● Does not directly show the overall model performance in a single value.
● Helps identify which classes are being confused.	● Can become difficult to interpret when comparing many experiments.
● Provides more detailed information than accuracy alone.	● Class imbalance, if present, can affect its interpretation.

7.5 ROC-AUC

ROC curves and AUC values were generated to evaluate the ability of the classifiers to distinguish between the different Wine classes. A multiclass **One-vs-Rest (OvR)** approach is used.

Succeeds / Fails

Where it succeeds	Where it fails
● Provides a useful measure of class discrimination.	● Multiclass ROC analysis is more complex than binary ROC.
● Allows comparison between different classifiers.	● AUC alone does not show the exact class-wise errors.
● Higher AUC generally indicates better class separation.	● ROC-AUC may not fully represent performance when the classes are unevenly distributed.

8. Wine Dataset PCA

8.1 PCA Analysis

Principal Component Analysis (PCA) is applied to reduce the dimensionality of the Wine dataset while preserving the most important information. The original features are transformed into a smaller number of principal components.

Succeeds / Fails

Where it succeeds	Where it fails
● Reduces the number of features.	● Some information may be lost during dimensionality reduction.
● Removes some redundancy between correlated features.	● Principal components are less interpretable than the original features.
● Can make the dataset easier to visualize and process.	● Too few components may reduce classification performance.

8.2 PCA + SVM

The PCA-reduced Wine dataset is given to the SVM classifier. The performance is compared with SVM trained on the original features.

Succeeds / Fails

Where it succeeds	Where it fails
● Reduces the feature space and computational complexity.	● Excessive dimensionality reduction can lower accuracy.
● SVM can perform effectively on the reduced feature set.	● Principal components do not have direct physical meaning.
● PCA may improve generalization by removing less useful information.	● Performance depends on the number of components selected.

8.3 PCA + MLP

The reduced features obtained from PCA are used as input to the MLP classifier. The model is trained and evaluated using the same performance metrics.

Succeeds / Fails

Where it succeeds	Where it fails
● Provides a smaller input space for the neural network.	● Important information can be lost if too few components are retained.
● Can reduce training complexity.	● MLP may require returning after PCA.
● May improve learning by removing redundant features.	● Reduced features may not always produce better accuracy.

8.4 PCA + Random Forest

Random Forest is applied to the PCA-transformed features, and its performance is compared with the model trained using the original Wine features.

Succeeds / Fails

Where it succeeds	Where it fails
● Reduces the number of input features.	● Random Forest can already handle the original 13 features effectively.
● Can lower computational requirements.	● PCA may remove features useful for tree-based classification.
● Provides a useful comparison of ensemble learning before and after PCA.	● PCA does not necessarily improve Random Forest accuracy.

9. Results

1. Untuned Results

	Dataset	Split	Classifier	Accuracy	Precision	Recall	F1 Score
0	Wine	50:50	SVM Linear	0.955056	0.955780	0.955056	0.954758
1	Wine	50:50	SVM Polynomial	0.853933	0.866553	0.853933	0.853971
2	Wine	50:50	SVM Gaussian	0.977528	0.977528	0.977528	0.977528
3	Wine	50:50	SVM Sigmoid	0.955056	0.956968	0.955056	0.955294
4	Wine	60:40	SVM Linear	0.958333	0.959974	0.958333	0.958336
5	Wine	60:40	SVM Polynomial	0.888889	0.902778	0.888889	0.889538
6	Wine	60:40	SVM Gaussian	0.986111	0.986574	0.986111	0.986041
7	Wine	60:40	SVM Sigmoid	1.000000	1.000000	1.000000	1.000000
8	Wine	70:30	SVM Linear	0.962963	0.963938	0.962963	0.962894
9	Wine	70:30	SVM Polynomial	0.907407	0.925214	0.907407	0.908485
10	Wine	70:30	SVM Gaussian	0.981481	0.982323	0.981481	0.981378
11	Wine	70:30	SVM Sigmoid	0.962963	0.965095	0.962963	0.962586
12	Wine	80:20	SVM Linear	0.944444	0.946581	0.944444	0.944269
13	Wine	80:20	SVM Polynomial	0.944444	0.951389	0.944444	0.943210
14	Wine	80:20	SVM Gaussian	0.972222	0.974074	0.972222	0.971970
15	Wine	80:20	SVM Sigmoid	1.000000	1.000000	1.000000	1.000000

	Dataset	Split	Classifier	Accuracy	Precision	Recall	F1 Score
0	Wine	50:50	MLP	0.966292	0.966861	0.966292	0.966392
1	Wine	60:40	MLP	1.000000	1.000000	1.000000	1.000000
2	Wine	70:30	MLP	0.981481	0.982639	0.981481	0.981554
3	Wine	80:20	MLP	1.000000	1.000000	1.000000	1.000000

	Dataset	Split	Classifier	Accuracy	Precision	Recall	F1 Score
0	Wine	50:50	Random Forest	0.977528	0.978340	0.977528	0.977404
1	Wine	60:40	Random Forest	0.986111	0.986667	0.986111	0.986131
2	Wine	70:30	Random Forest	1.000000	1.000000	1.000000	1.000000
3	Wine	80:20	Random Forest	1.000000	1.000000	1.000000	1.000000

	Dataset	Split	Classifier	Accuracy	Precision	Recall	F1 Score
0	Digits	50:50	SVM Linear	0.973665	0.973832	0.973665	0.973687
1	Digits	50:50	SVM Polynomial	0.974377	0.976763	0.974377	0.974915
2	Digits	50:50	SVM Gaussian	0.981495	0.981805	0.981495	0.981535
3	Digits	50:50	SVM Sigmoid	0.943060	0.944040	0.943060	0.943263
4	Digits	60:40	SVM Linear	0.974644	0.974765	0.974644	0.974638
5	Digits	60:40	SVM Polynomial	0.977758	0.979585	0.977758	0.978163
6	Digits	60:40	SVM Gaussian	0.985320	0.985475	0.985320	0.985335
7	Digits	60:40	SVM Sigmoid	0.945730	0.946305	0.945730	0.945877
8	Digits	70:30	SVM Linear	0.978648	0.978933	0.978648	0.978676
9	Digits	70:30	SVM Polynomial	0.976868	0.979458	0.976868	0.977428
10	Digits	70:30	SVM Gaussian	0.986358	0.986450	0.986358	0.986354
11	Digits	70:30	SVM Sigmoid	0.941874	0.942647	0.941874	0.942059
12	Digits	80:20	SVM Linear	0.978648	0.978782	0.978648	0.978669
13	Digits	80:20	SVM Polynomial	0.976868	0.978898	0.976868	0.977336
14	Digits	80:20	SVM Gaussian	0.986655	0.986882	0.986655	0.986668
15	Digits	80:20	SVM Sigmoid	0.943060	0.943695	0.943060	0.943174

	Dataset	Split	Classifier	Accuracy	Precision	Recall	F1 Score
0	Digits	50:50	MLP	0.977580	0.977592	0.977580	0.977573
1	Digits	60:40	MLP	0.979093	0.979124	0.979093	0.979096
2	Digits	70:30	MLP	0.981020	0.981121	0.981020	0.981041
3	Digits	80:20	MLP	0.982206	0.982353	0.982206	0.982233

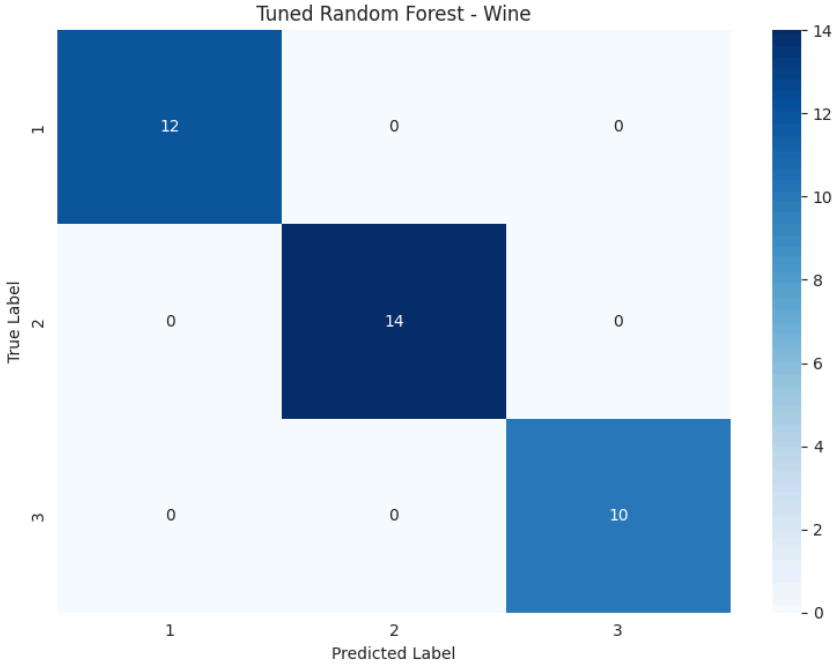
	Dataset	Split	Classifier	Accuracy	Precision	Recall	F1 Score
0	Digits	50:50	Random Forest	0.979359	0.979416	0.979359	0.979365
1	Digits	60:40	Random Forest	0.985765	0.985800	0.985765	0.985763
2	Digits	70:30	Random Forest	0.985172	0.985224	0.985172	0.985172
3	Digits	80:20	Random Forest	0.986655	0.986747	0.986655	0.986629

2. Tuned Results

```
Best Random Forest parameters:
{'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}

Best CV Accuracy:
0.9862068965517242
```

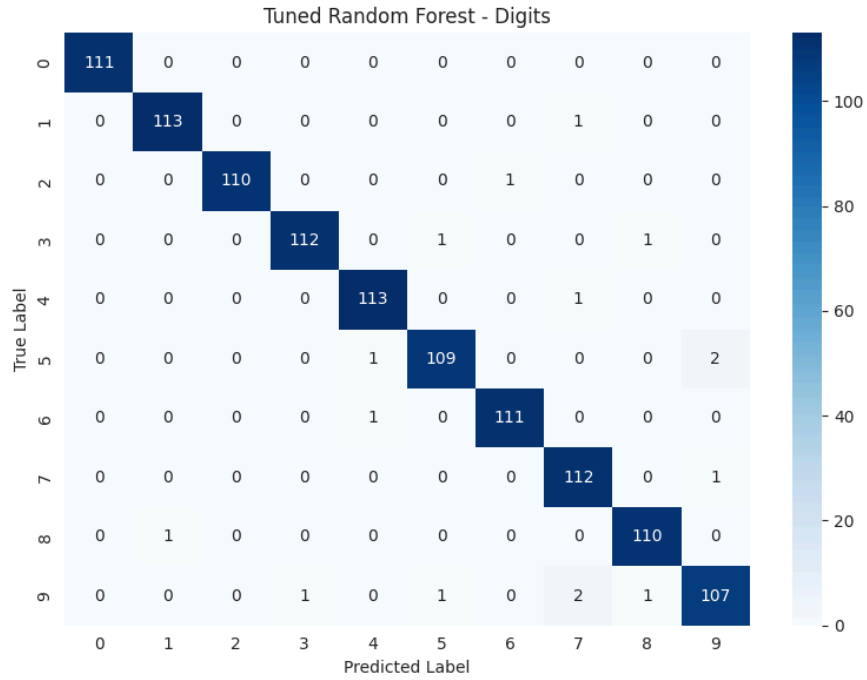
```
Tuned Random Forest Results
------------------------
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0
```




```
Best Random Forest parameters for Digits:
{'max_depth': None, 'max_features': 'log2', 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 200}

Best CV Accuracy for Digits:
0.9799819075348557

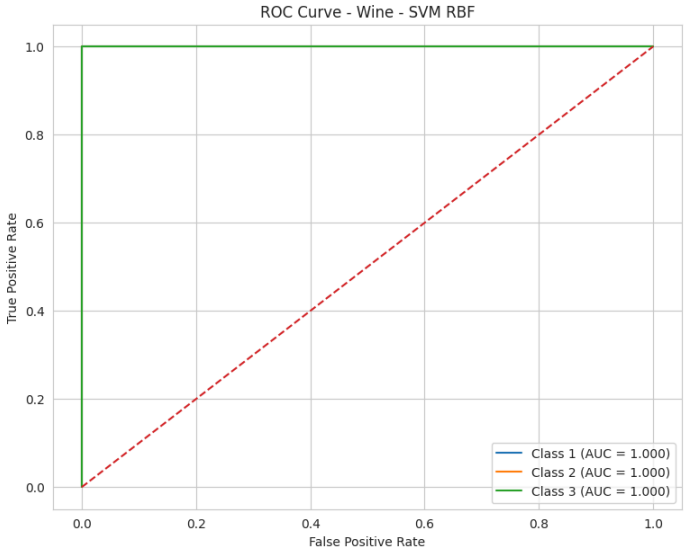
Tuned Random Forest Results on Digits
-------------------------------------
Accuracy: 0.9857651245551602
Precision: 0.9858160886948233
Recall: 0.9857651245551602
F1 Score: 0.9857561482171892
```

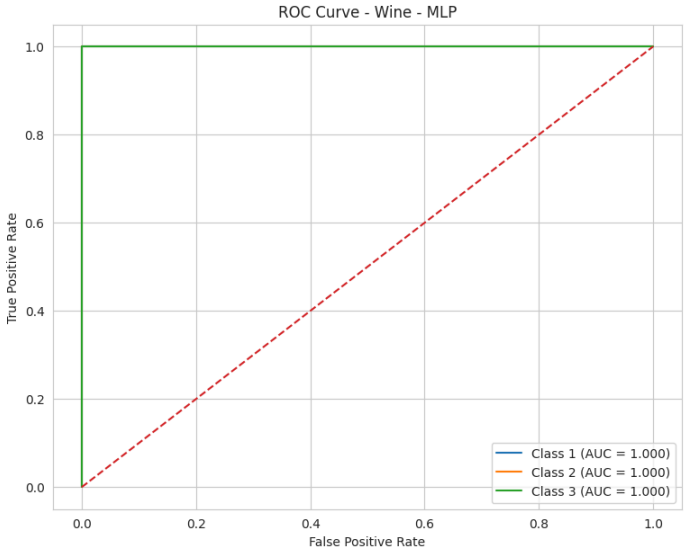


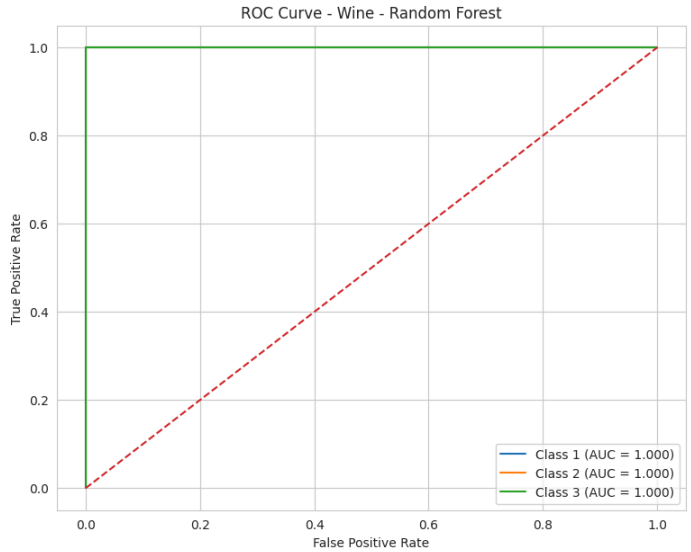
3. Train-Test Split Comparison

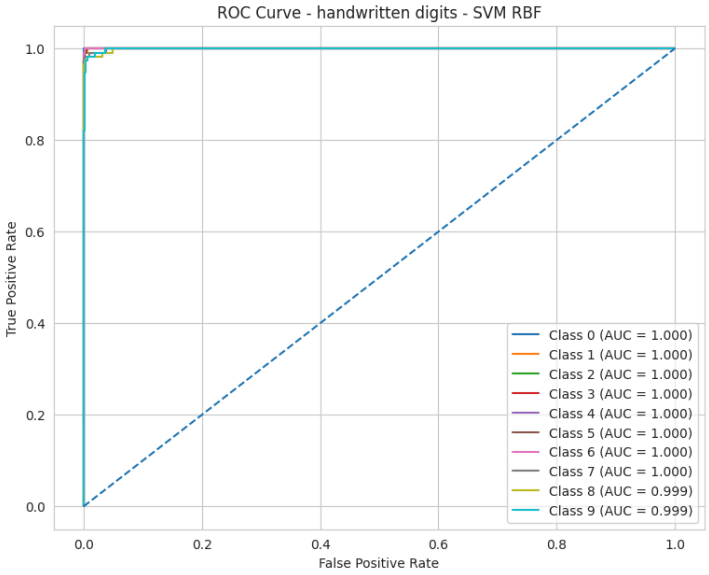
	Dataset	Split	Classifier	Accuracy	Precision	Recall	F1 Score
0	Wine	50:50	SVM Linear	0.955056	0.955780	0.955056	0.954758
1	Wine	50:50	SVM Polynomial	0.853933	0.866553	0.853933	0.853971
2	Wine	50:50	SVM Gaussian	0.977528	0.977528	0.977528	0.977528
3	Wine	50:50	SVM Sigmoid	0.955056	0.956968	0.955056	0.955294
4	Wine	60:40	SVM Linear	0.958333	0.959974	0.958333	0.958336
5	Wine	60:40	SVM Polynomial	0.888889	0.902778	0.888889	0.889538
6	Wine	60:40	SVM Gaussian	0.986111	0.986574	0.986111	0.986041
7	Wine	60:40	SVM Sigmoid	1.000000	1.000000	1.000000	1.000000
8	Wine	70:30	SVM Linear	0.962963	0.963938	0.962963	0.962894
9	Wine	70:30	SVM Polynomial	0.907407	0.925214	0.907407	0.908485
10	Wine	70:30	SVM Gaussian	0.981481	0.982323	0.981481	0.981378
11	Wine	70:30	SVM Sigmoid	0.962963	0.965095	0.962963	0.962586
12	Wine	80:20	SVM Linear	0.944444	0.946581	0.944444	0.944269
13	Wine	80:20	SVM Polynomial	0.944444	0.951389	0.944444	0.943210
14	Wine	80:20	SVM Gaussian	0.972222	0.974074	0.972222	0.971970
15	Wine	80:20	SVM Sigmoid	1.000000	1.000000	1.000000	1.000000
16	Wine	50:50	MLP	0.966292	0.966861	0.966292	0.966392
17	Wine	60:40	MLP	1.000000	1.000000	1.000000	1.000000
18	Wine	70:30	MLP	0.981481	0.982639	0.981481	0.981554
19	Wine	80:20	MLP	1.000000	1.000000	1.000000	1.000000
20	Wine	50:50	Random Forest	0.977528	0.978340	0.977528	0.977404
21	Wine	60:40	Random Forest	0.986111	0.986667	0.986111	0.986131
22	Wine	70:30	Random Forest	1.000000	1.000000	1.000000	1.000000
23	Wine	80:20	Random Forest	1.000000	1.000000	1.000000	1.000000
24	Digits	50:50	SVM Linear	0.973665	0.973832	0.973665	0.973687
25	Digits	50:50	SVM Polynomial	0.974377	0.976763	0.974377	0.974915
26	Digits	50:50	SVM Gaussian	0.981495	0.981805	0.981495	0.981535
27	Digits	50:50	SVM Sigmoid	0.943060	0.944040	0.943060	0.943263
28	Digits	60:40	SVM Linear	0.974644	0.974765	0.974644	0.974638
29	Digits	60:40	SVM Polynomial	0.977758	0.979585	0.977758	0.978163
30	Digits	60:40	SVM Gaussian	0.985320	0.985475	0.985320	0.985335
31	Digits	60:40	SVM Sigmoid	0.945730	0.946305	0.945730	0.945877
32	Digits	70:30	SVM Linear	0.978648	0.978933	0.978648	0.978676
33	Digits	70:30	SVM Polynomial	0.976868	0.979458	0.976868	0.977428
34	Digits	70:30	SVM Gaussian	0.986358	0.986450	0.986358	0.986354
35	Digits	70:30	SVM Sigmoid	0.941874	0.942647	0.941874	0.942059
36	Digits	80:20	SVM Linear	0.978648	0.978782	0.978648	0.978669
37	Digits	80:20	SVM Polynomial	0.976868	0.978898	0.976868	0.977336
38	Digits	80:20	SVM Gaussian	0.986655	0.986882	0.986655	0.986668
39	Digits	80:20	SVM Sigmoid	0.943060	0.943695	0.943060	0.943174
40	Digits	50:50	MLP	0.977580	0.977592	0.977580	0.977573
41	Digits	60:40	MLP	0.979093	0.979124	0.979093	0.979096
42	Digits	70:30	MLP	0.981020	0.981121	0.981020	0.981041
43	Digits	80:20	MLP	0.982206	0.982353	0.982206	0.982233
44	Digits	50:50	Random Forest	0.979359	0.979416	0.979359	0.979365
45	Digits	60:40	Random Forest	0.985765	0.985800	0.985765	0.985763
46	Digits	70:30	Random Forest	0.985172	0.985224	0.985172	0.985172
47	Digits	80:20	Random Forest	0.986655	0.986747	0.986655	0.986629

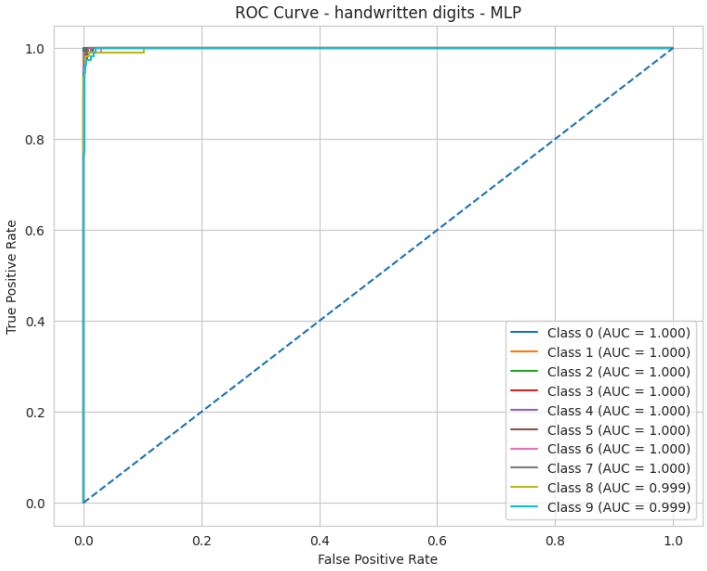
4. ROC-AUC

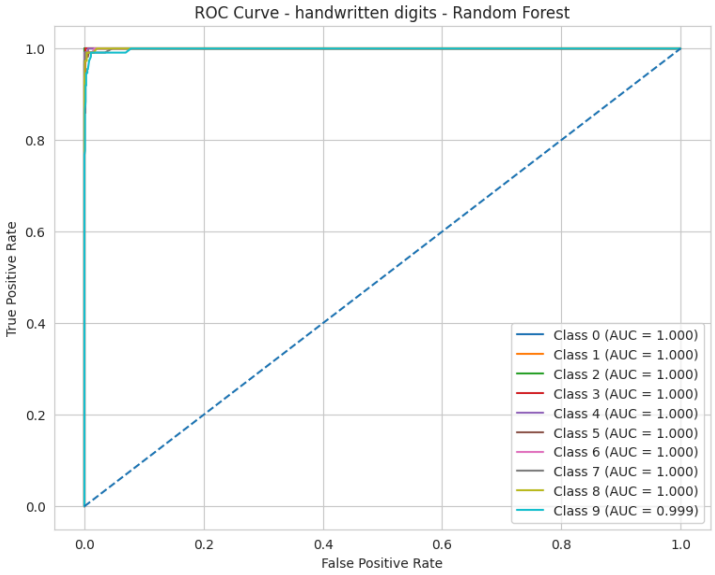

ROC Curve - Wine - SVM RBF
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
Class 1 (AUC = 1.000)
Class 2 (AUC = 1.000)
Class 3 (AUC = 1.000)


ROC Curve - Wine - MLP
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
Class 1 (AUC = 1.000)
Class 2 (AUC = 1.000)
Class 3 (AUC = 1.000)


ROC Curve - Wine - Random Forest
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
Class 1 (AUC = 1.000)
Class 2 (AUC = 1.000)
Class 3 (AUC = 1.000)


ROC Curve - handwritten digits - SVM RBF
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
Class 0 (AUC = 1.000)
Class 1 (AUC = 1.000)
Class 2 (AUC = 1.000)
Class 3 (AUC = 1.000)
Class 4 (AUC = 1.000)
Class 5 (AUC = 1.000)
Class 6 (AUC = 1.000)
Class 7 (AUC = 1.000)
Class 8 (AUC = 0.999)
Class 9 (AUC = 0.999)


ROC Curve - handwritten digits - MLP
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
Class 0 (AUC = 1.000)
Class 1 (AUC = 1.000)
Class 2 (AUC = 1.000)
Class 3 (AUC = 1.000)
Class 4 (AUC = 1.000)
Class 5 (AUC = 1.000)
Class 6 (AUC = 1.000)
Class 7 (AUC = 1.000)
Class 8 (AUC = 0.999)
Class 9 (AUC = 0.999)


ROC Curve - handwritten digits - Random Forest
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
Class 0 (AUC = 1.000)
Class 1 (AUC = 1.000)
Class 2 (AUC = 1.000)
Class 3 (AUC = 1.000)
Class 4 (AUC = 1.000)
Class 5 (AUC = 1.000)
Class 6 (AUC = 1.000)
Class 7 (AUC = 1.000)
Class 8 (AUC = 1.000)
Class 9 (AUC = 0.999)

5. Handwritten Digits PCA

	Dataset	Classifier	Accuracy	Precision	Recall	F1 Score
0	Wine	PCA + SVM	0.972222	0.974074	0.972222	0.971970
1	Wine	PCA + MLP	0.972222	0.974074	0.972222	0.971970
2	Wine	PCA + Random Forest	0.944444	0.944444	0.944444	0.944444
3	Digits	PCA + SVM	0.986655	0.986882	0.986655	0.986668
4	Digits	PCA + MLP	0.976868	0.977071	0.976868	0.976869
5	Digits	PCA + Random Forest	0.970641	0.971062	0.970641	0.970697

10. Overall Results Comparison

	Model Name	Best Parameters	Accuracy	Precision	Recall	F1 Score
0	SVM (Wine)	{'C': 1, 'gamma': 0.01, 'kernel': 'rbf'}	98.61	98.66	98.61	98.60
1	MLP (Wine)	{'hidden_layer_sizes': (50,), 'learning_rate_i...	100.00	100.00	100.00	100.00
2	Random Forest (Wine)	{'max_depth': None, 'max_features': 'sqrt', 'm...	100.00	100.00	100.00	100.00
3	SVM (Digits)	{'kernel': 'rbf', 'probability': True, 'random...	98.67	98.69	98.67	98.67
4	MLP (Digits)	{'hidden_layer_sizes': (100,), 'activation': '...	98.22	98.24	98.22	98.22
5	Random Forest (Digits)	{'max_depth': None, 'max_features': 'log2', 'm...	98.67	98.67	98.67	98.66

11. Discussion

The experiments show that SVM, MLP, and Random Forest can effectively classify both the Wine and Handwritten Digits datasets. Performance varied with the train-test split and hyperparameter settings. Parameter tuning generally improved model performance. PCA reduced the number of features while maintaining good classification accuracy. Overall, the models achieved strong results, with the best models reaching the required 90%+ accuracy.

12. Conclusion

The project successfully implemented and compared SVM (Linear, Polynomial, Gaussian/RBF, and Sigmoid), MLP, and Random Forest on both UCI datasets. Accuracy, Precision, Recall, F1-score, confusion matrices, ROC-AUC, parameter tuning, and PCA were evaluated. The experiments demonstrate that machine learning classifiers can achieve high accuracy and effectively classify both wine categories and handwritten digits.
